

SUMMARY

보안 SIEM 제품에서 백엔드, 데이터 파이프라인, Kubernetes 운영 인프라를 함께 다뤄 온 백엔드 엔지니어입니다.

Go 기반 멀티테넌트 수집 플랫폼을 0→1로 구축하고, Kafka/Redis Streams/PostgreSQL/OpenSearch를 이용해 데이터 수집, 정규화 흐름을 운영했습니다.

다양한 기능 구현과 함께 CI/CD를 통한 개발 속도 개선, 운영 문서화까지 제품 운영에 필요한 기반을 함께 개선해 왔습니다.

1. SENTIVEX SIEM

(주)인브릿지 | Backend Engineer

2025.06 - 현재

프로젝트 개요: 보안 벤더(CrowdStrike, SentinelOne, Symantec, Palo Alto 등)의 침해 로그를 수집·정규화하고, OpenSearch 기반 탐지·분석과 AI 보고서 생성을 제공하는 SIEM 플랫폼.

역할: 백엔드 및 데이터 파이프라인 설계와 구축, AWS EKS 구축 및 운영, CI/CD 및 운영 안정화 주도.

기술 스택: Next.js, NestJS, Go, PostgreSQL, Redis, Kafka, OpenSearch, AWS EKS

대표 성과:

- 인증, 테넌트, 운영 API를 중심으로 수집/정규화 로직과 AI 로직 등 확장 가능한 백엔드 Hub & Spoke 아키텍처 설계
- 마이크로서비스 기반 수집 플랫폼 구축하고, Adapter 패턴을 기반으로 신규 벤더 추가 비용을 축소
- CrowdStrike, SentinelOne, Symantec 등 멀티벤더 수집기를 운영하며 20+ API 엔드포인트를 연동하고, Worker Heartbeat와 self-healing으로 50일+ 무중단 수집 운영
- 7노드 AWS EKS 운영 클러스터와 CloudNativePG HA, Strimzi Kafka KRaft, SealedSecret, ALB/Route53 기반 운영 인프라 구축
- 5개 서비스 레포가 공유하는 platform-ci를 설계해 reusable workflow/composite action/OIDC 기반 배포로 전환하고 4개월 유지

1.1 백엔드 Hub & Spoke 아키텍처 설계

문제: SIEM 플랫폼에는 인증, 테넌트 관리, 운영 API, 벤더 수집, 이벤트 정규화, AI 보고서 생성처럼 성격이 다른 기능이 함께 필요했습니다. 이를 단일 백엔드에 계속 누적하면 공통 정책과 도메인 로직이 기능별 구현에 흩어지고, 반대로 서비스를 너무 잘게 나누면 개발 속도가 느려질 위험이 있었습니다.

접근:

- 인증, 테넌트 컨텍스트, 권한, 운영 API, 공통 도메인 계약을 Hub 역할로 묶고, 수집/정규화/AI 보고서처럼 변화가 큰 기능은 Spoke로 분리
- Hub는 NestJS 기반 API 계층과 공통 도메인 모델을 중심으로 구성하고, Spoke는 Go 수집 런타임과 파이썬 AI/분석 워크플로처럼 독립적으로 확장 가능한 실행 단위로 설계
- 공통 인증, 테넌트, 운영 정책은 Hub에서 일관되게 적용되도록 설계

결과:

- Hub의 Swagger 문서를 통해 프론트엔드의 연동 지점을 통합하고, 백엔드 구현 변경이 프론트엔드에 미치는 영향을 줄임
- 신규 벤더, 정규화 파이프라인, AI 기능을 기존 API/테넌트 정책을 흔들지 않고 Spoke 단위로 추가할 수 있는 구조 확보
- 공통 기능을 Hub에 집중시켜 Spoke별 중복 구현을 줄이고, 운영 정책 적용 지점을 통합
- NestJS API, Go 수집 런타임, AI/분석 워크플로의 경계를 분리해 기능 변경 시 영향 범위와 배포 리스크를 축소

1.2 멀티테넌트 보안 데이터 수집 플랫폼 0→1

문제: 벤더별 API, polling/streaming 방식, 테넌트 격리, checkpoint, retry 정책이 섞여 있어 신규 벤더를 추가할 때마다 수집 로직이 중복되고 운영 기준이 흔들릴 위험이 있었습니다.

접근:

- Control Plane(api-server), scheduling, worker, stream supervisor로 분리한 4개 마이크로서비스 구조로 설계
- **Polling**, **Trigger**, **Streaming** 인터페이스 계약을 정의하고, 벤더별 구현이 이를 따르도록 하여 수집 방식 차이를 세개의 실행 모델로 통합
- PostgreSQL에 상태와 정책, Redis Streams에는 실행 큐, Kafka에는 downstream 이벤트를 맡겨 책임 경계를 분리

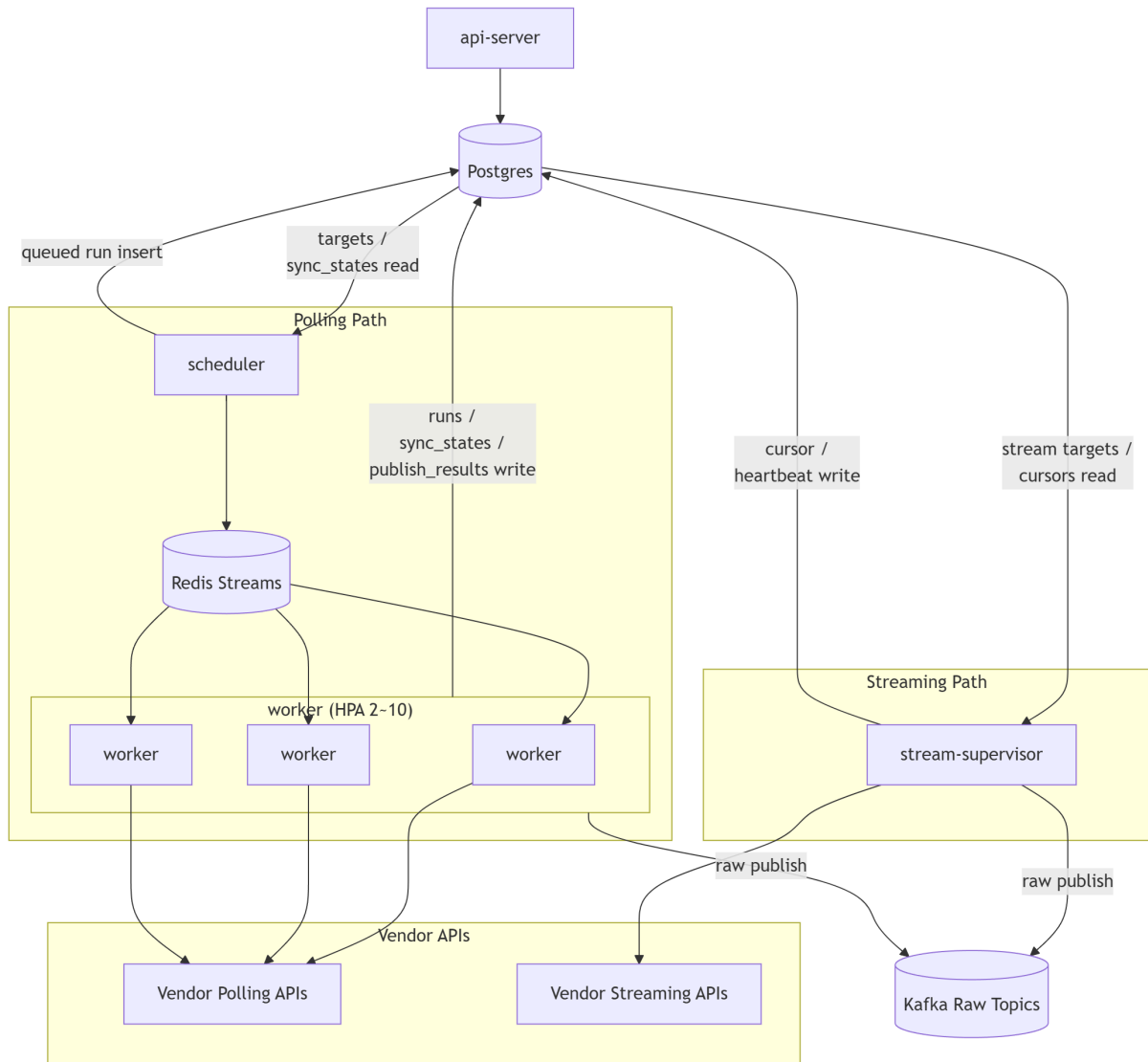


그림 1: 멀티테넌트 보안 데이터 수집 플랫폼 아키텍처

결과:

- CrowdStrike polling/streaming을 시작으로 SentinelOne, Symantec 연동까지 확장 가능한 구조 확보
- 9개 핵심 도메인 엔터티(tenant, vendor, credential, integration, target, sync state, run, publish result, stream offset)를 단일 모델로 정리
- 벤더별 API 차이를 Adapter 패턴으로 격리해 신규 벤더 추가 시 기존 로직 변경과 리스크를 축소

1.3 AWS EKS 운영 인프라와 공용 CI/CD 플랫폼

문제: 온프레미스 중심 운영과 서비스별 개별 배포 방식은 장애 대응, 권한 관리, 배포 재현성 측면에서 한계가 있었습니다. 특히 Git 평문 secret, self-hosted runner 환경 차이가 운영 리스크로 이어질 수 있었습니다.

접근:

- 7노드 EKS 클러스터를 system/app/data 계층으로 분리하고, ALB/Route53/ACM/ExternalDNS/cert-manager/SealedSecrets를 표준 플랫폼 계층으로 구성
- CloudNativePG 3-instance HA, Strimzi Kafka KRaft(controller 3 + broker 3), OpenSearch Security plugin/mTLS, ElastiCache Valkey를 운영 데이터 계층으로 배치
- 5개 서비스 레포가 공유하는 `platform-ci`를 reusable workflow + composite action 이중 레이어로 설계

결과:

- PostgreSQL single instance 구조를 3-instance HA로 전환하고 failover 약 10초 수준의 운영 기반 확보
- 서비스 5개 x 환경 3개의 개별 workflow 관리 부담을 3개 reusable workflow + 3개 action으로 축소
- SealedSecret 운영 절차와 GitOps 책임 경계를 문서화하고 공유함. git에 평문 secret을 올리는 보안 문제를 0건을 유지

1.4 Production Incident / SRE 사례

문제: 운영 환경에서는 컴포넌트 하나의 장애가 다른 계층의 알람으로 나타나거나, 단순 재시작으로는 해결되지 않는 병목이 반복됐습니다. 장애를 복구하는 것과 재발을 줄이는 절차를 진행해야 했습니다.

대표 사례:

- Loki 로그 드랍 알람: Loki 설정을 올리지 않고 로그 폭주 원인을 역추적함. OpenSearch JVM heap OOM이 원인임을 확인하고 무중단 복구 조치
- OpenSearch 샤딩 근본 원인 분석: 거대 단일 샤드와 과도한 일별 인덱스 분할이 복구 비용을 키우는 구조를 확인하고, 용량 기반 rollover가 가능한 data stream 구조로 재설계 방향 제시
- CloudNativePG PgBouncer Pooler: 요청을 바로 적용하지 않고 데이터베이스 커넥션을 확인한 뒤, 기존 서비스 경로를 유지한 채 PgBouncer로 점진적 전환

2. MICROSOFT TEAMS ADD-IN

(주)인브리치 | Internship

2025.03 - 2025.05

프로젝트 개요: Microsoft Teams 통화·미팅 자동 녹취 및 LLM 요약 Add-in.

역할: 중도 합류 후 어드민 페이지, 백엔드 API, 협업 체계 정리 담당.

기술 스택: React, JavaScript, Node.js, Express, ASP .NET, PostgreSQL(Supabase), Docker, GitHub

대표 성과:

- 프로덕션 서버 직접 수정 방식에서 GitHub 브랜치/PR 기반 협업 체계로 전환해 코드 덮어쓰기와 충돌로 발생하던 일일 장애를 제거
- Docker 기반 개발 환경을 도입해 OS별 실행 차이를 줄이고 로컬 디버깅 재현성을 개선
- Caller/Callee 구조를 user/counterpart + direction 플래그 모델로 단순화하는 RFC를 작성하고, API 정렬·페이지네이션 규칙을 표준화
- React 어드민 화면과 Node.js/Express API를 개발하며 프론트엔드와 백엔드 간 데이터 계약을 정리

2.1 RFC 작성과 API/데이터 모델 정리

문제: 통화 녹취 데이터 모델이 Caller/Callee 구조로 되어 있어, 통화 방향과 상대방 구분이 API와 프론트엔드에서 일관성 없이 처리되고 있었습니다. 또한 배포중인 서버 직접 수정 방식으로 개발 협업이 이루어지면서 코드 덮어쓰기와 충돌로 인한 일일 장애가 발생하고 있었습니다.

접근:

- Caller/Callee 구조가 통화 방향과 상대방 구분을 혼재시키는 문제를 RFC로 제기하고, user/counterpart + direction 플래그 모델로 단순화하는 제안 작성

- API 엔드포인트와 페이지네이션 규칙을 표준화해 프론트엔드와 백엔드 간 데이터 계약을 명확히 정리
- 배포중인 서버 직접 수정 방식에서 GitHub 브랜치/PR 기반 협업 체계로 전환해 코드 덮어쓰기와 충돌로 발생하던 일일 장애를 제거
- Docker 기반 개발 환경을 도입해 OS별 실행 차이를 줄이고 로컬 디버깅 재현성을 개선

결과:

- 통화 방향과 상대방 구분이 명확해지고, API와 프론트엔드 간 데이터 계약이 정리되어 개발 효율성 향상
- 코드 덮어쓰기와 충돌로 인한 일일 장애가 제거되고, 안정적인 개발 협업 체계 구축
- OS별 실행 차이가 줄어들고, 로컬 디버깅 재현성이 개선되어 개발 속도 향상